TeamID 03

Name	Roll no.
Ayush Kumar Dubey	20251651030
Abhinav Jain	20251651003
Anuj Gupta	20251651025
Bharath Kumar MP	20251651031
Kaushik Nanda Upadhaya	20251651050

Task 1:

Pick the partner edge and justify SOAP

Context

- **Partner:** CampusPay Payment Gateway
- **Operation:** `charge`
- CampusEats integrates with CampusPay to process payments for student food orders.
- SOAP is selected because payment processing requires:
  - A strong WSDL-based contract
  - Standardized fault handling
  - Message-level security
  - Enterprise transaction support
- The formal SOAP contract clearly defines the messages, responses, and errors exchanged between CampusEats and the payment gateway.
- The rest of CampusEats uses REST because normal application APIs can use lightweight HTTP/JSON communication without the additional complexity of SOAP.

Task 4:

Show the HTTP binding

This section shows the raw HTTP POST that carries the `charge` SOAP request (`soap-request.xml`) to the CampusPay Payment Gateway. Every value below is taken directly from `partner.wsdl`'s `<service>`/`<port>` and `<binding>` elements.

- **Method:** `POST` — required by the WSDL's `soap:binding` transport (SOAP over HTTP).

- **Host / path:** taken from `<soap:address location="https://sandbox.payments.example.com/campuseats/PaymentGateway"/>`
- **SOAPAction:** taken from `<soap:operation soapAction="https://payments.campuseats.example.com/soap/charge"/>` on the `charge` operation.
- **Content-Type:** `text/xml; charset="utf-8"` — standard for SOAP 1.1, matching the envelope namespace used in `soap-request.xml`.
- **Content-Length:** `1689` — exact byte size of `soap-request.xml`.

HTTP request block

```
POST /campuseats/PaymentGateway HTTP/1.1
Host: sandbox.payments.example.com
Content-Type: text/xml; charset="utf-8"
Content-Length: 1689
SOAPAction: "https://payments.campuseats.example.com/soap/charge"

<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:tns="https://payments.campuseats.example.com/soap"
               xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-
200401-wss-wssecurity-secext-1.0.xsd"
               xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-
200401-wss-wssecurity-utility-1.0.xsd">
    <soap:Header>
        <wsse:Security soap:mustUnderstand="1">
            <wsu:Timestamp wsu:Id="TS-1">
                <wsu:Created>2026-08-26T09:15:00Z</wsu:Created>
                <wsu:Expires>2026-08-26T09:20:00Z</wsu:Expires>
            </wsu:Timestamp>
            <wsse:UsernameToken wsu:Id="UT-1">
                <wsse:Username>CE-MERCHANT-4471</wsse:Username>
                <wsse:Password Type="http://docs.oasis-
open.org/wss/2004/01/oasis-200401-wss-username-token-profile-
1.0#PasswordDigest">Xk3f9s0aP2m7QeH1v9c0jklm3Rk=</wsse:Password>
                <wsse:Nonce EncodingType="http://docs.oasis-
open.org/wss/2004/01/oasis-200401-wss-soap-message-security-
1.0#Base64Binary">MTIzNDU2Nzg5MA==</wsse:Nonce>
                <wsu:Created>2026-08-26T09:15:00Z</wsu:Created>
            </wsse:UsernameToken>
        </wsse:Security>
    </soap:Header>
    <soap:Body>
        <tns:ChargeRequest>
            <tns:orderId>ORD-20260826-0091</tns:orderId>
            <tns:idempotencyKey>idem_7f3a9c21e4</tns:idempotencyKey>
            <tns:amountMinorUnits>30000</tns:amountMinorUnits>
            <tns:currency>INR</tns:currency>
            <tns:cardToken>tok_9f2c</tns:cardToken>
```

```
        </tns:ChargeRequest>
    </soap:Body>
</soap:Envelope>
```

Task 5:

Describe discovery (modern form)

Discovery

CampusEats does not run its own UDDI server. Instead, discovery happens the way real partner integrations work today: CampusPay publishes its WSDL at a stable, versioned URL on its own developer/sandbox domain, and CampusEats' integration team retrieves it directly rather than querying a live registry at runtime.

- **How CampusEats obtains the WSDL:** direct URL lookup — `https://sandbox.payments.example.com/campuseats/PaymentGateway?wsdl`. This is the same endpoint declared in `partner.wsdl`'s `<soap:address location="...">`, with the conventional `?wsdl` query parameter that most SOAP servers use to expose their own contract.
- **Why not a live registry:** UDDI-style dynamic lookup at request time went out of common use; the "modern form" of discovery is a one-time, human-curated catalogue entry — checked in alongside the code — that records where the contract lives. Any change to the partner's WSDL is picked up by updating this one entry, not by querying a directory service on every call.
- **Where this record lives in practice:** a service catalogue (e.g. an internal developer portal, a `partners/` folder in the repo, or a tool like Backstage) that lists every external dependency CampusEats has, one row per partner operation.

Registry entry (tModel-style pointer)

Field	Value
Business	CampusPay Payment Gateway
Service	PaymentGatewayService — `charge` operation
Endpoint	`https://sandbox.payments.example.com/campuseats/PaymentGateway`
WSDL (tModel pointer)	`https://sandbox.payments.example.com/campuseats/PaymentGateway?wsdl`
Binding	`PaymentGatewaySoapBinding` (SOAP 1.1 over HTTP, document style)
Namespace	`https://payments.campuseats.example.com/soap`

This single record plays the same role a UDDI `businessEntity` + `tModel` pair used to play — it tells any CampusEats service exactly which business it's calling, which contract that business speaks, and where to fetch that contract from, without needing a live registry to resolve it at runtime.

Task 6

Fault Mapping

Overview

The integration layer translates partner-specific SOAP faults into the error codes defined by our `placeOrder` contract.

The partner's internal fault names are **not exposed to students or clients**. This keeps the partner's vocabulary isolated from our own API contract.

Fault Mapping Table

Partner SOAP Fault	Our `placeOrder` Error	Meaning
`card_declined`	`PAYMENT_DECLINED`	The payment was rejected
`payment_failed`	`PAYMENT_FAILED`	Payment processing failed
`item_not_found`	`ITEM_NOT_FOUND`	A requested food item does not exist
`item_unavailable`	`ITEM_UNAVAILABLE`	A requested food item is currently unavailable
`invalid_quantity`	`INVALID_QUANTITY`	The requested quantity is invalid
`invalid_address`	`INVALID_ADDRESS`	The delivery address is invalid or unavailable
`no_rider_available`	`NO_RIDER_AVAILABLE`	A rider cannot currently be assigned
`empty_cart`	`EMPTY_CART`	No food items were provided

Example

If the partner SOAP service returns:

```
<soap:Fault>
    <faultcode>soap:Client</faultcode>
    <faultstring>card_declined</faultstring>
</soap:Fault>
```